Instructions to Calibrate Cameras and Hand tracking

1) Open a Console inside HandTracking
2) In “calibrationCameras_settings.yaml”: modify the parameters for calibrating the two cameras.
    a. IDs of cameras
    b. Frame width and height (we want 1920 x 1080 with our webcams)
    c. Number of frames to capture for calibration (10 is ok)
    d. Size of checkboard square (not mandatory)
    e. Checkboard rows and columns: **CAREFUL**, if your checkboard is 7x10 you need to write 6x9 -> it’s the length – 1
    f. Cooldown: delay between frame captures
3) Write “python .\calibrationCameras.py .\calibrationCameras_settings.yaml”
4) The first camera should turn on
5) Follow the instructions for the single camera calibration.
6) The second camera should turn on
7) Follow the instructions for the single camera calibration.
8) Verify the frames captured for the calibration: follow the instructions (space to validate a frame, s to remove it)
9) The two cameras should turn on simultaneously
10) Follow the instructions for the multiple cameras calibration.
11) Verify the frames captured for the calibration: follow the instructions (space to validate a frame, s to remove it)
12) When finished, the cameras will be on, showing their common 3D coordinate system. Quit using Esc.
13) “camera_parameters” stores the camera parameters of the position of the cameras. **Note:** if you move them you should repeat the previous steps

The calibration of the cameras is ready to be used. Now we need to calibrate the environment for hand tracking (changing base for digital environment).

14) Open a Console inside HandTracking
15) Execute “python .\calibrationEnvironment.py A B”: (follow same order as in the calibrationCamerasSettings.yaml)
    a. A is the number of the first camera
    b. B is the number of the second camera
16) We will now draw a coordinate system, first (0,0), then (1,0), (0,1) and height.
17) Place your index fingertip at your origin in the physical world. Press space.
18) Place the physical coordinate system at your origin.
19) Place your index fingertip at the x-axis. Press space.
20) Place your index fingertip at the y-axis. Press space.
21) Place your index fingertip at the z-axis. Press space.
22) Quit the python script using Esc.

Calibration of new 3D coordinates is done. To use them, we now need to coordinate Unity, the ESP and this calibration.

23) Connect an ESP32 to your computer. Open HandTracking/UDPServer.ino
24) Upload the script on your ESP. Make sure the port is the same in the ino script, the python handpose3d.py and the Unity script HandData.cs.

Ensure the computer with Unity and the computer tracking the hand are both connected to the same ESP with the same direction and port.

25) Open a Console inside HandTracking
26) Execute “python .\handpose3d.py A B”: (follow same order as in the calibrationCamerasSettings.yaml)
27) Three vector magnitudes are printed (x, y, z). Enter them in your Unity program before compiling. In HandData.cs: normOrigin in Inspector (e.g., normOrigin = new Vector3(1.83, 1.88, 1.78)).

The tracking of the hand will now be done in real time, and all phalanges position sent.

28) In HandData.cs: update the coordinate system size you’ve used for calibration in coordPhysical (e.g., coordPhysical = new Vector3(0.05f, 0.05f, 0.05f))
29) In HandData.cs: update your environment size in realEnv (e.g., realEnv = new Vector3(0.2, 0.08f, 0.2f)) If you want a real-life scale, this vector should be 0.1, 0.1, 0.1.
30) Make sure the HandPosition game object is located at the origin of your digital environment (e.g., for the Voxon, you want to place it at half of the bottom corner of the view finder; at a height of 0 as the Voxon goes up and down). The coordinate system from HandsUpdate should be the same as the one you’re using in real life.
31) You can now start the game on Unity and see your hands moving. You can also compile the APK and see the hands in your compiled game.